



Patrones estructurales: cómo se arman objetos con objetos

Los patrones estructurales

Los patrones creacionales resolvieron quién escribe el *new*. Los estructurales se ocupan de lo que viene después, que es cómo se combinan clases y objetos para formar estructuras más grandes sin que el código que las usa se vuelva más complicado. Un dibujo hecho de figuras y de grupos de figuras, un personaje con su equipamiento, una operación que necesita cinco servicios distintos para completarse. En los tres casos hay varios objetos que tienen que actuar como uno.

La herramienta que casi todos los patrones estructurales usan es la composición, un objeto que contiene a otro y le delega parte del trabajo. La herencia también sirve para extender una clase, pero decide en tiempo de compilación y de a una clase por vez. La composición decide en tiempo de ejecución, se puede apilar y se puede deshacer. Por eso el consejo más repetido del libro de Gamma y compañía es preferir la composición a la herencia, y los tres patrones de esta pre clase son tres maneras de aplicarlo.

Esta pre clase presenta Composite, para tratar un objeto y un grupo de objetos de la misma manera; Decorator, para agregar responsabilidades a un objeto sin cambiar su clase; y Facade, para ofrecer una entrada simple a un conjunto de clases complicado. Cada uno se describe igual que en la pre clase anterior, con el problema que resuelve, la solución en general con sus participantes, un ejemplo, y sus ventajas y desventajas. La referencia más accesible sigue siendo Refactoring Guru (<https://refactoring.guru/es/design-patterns>), con los siete estructurales del catálogo explicados con diagramas y código en Java.

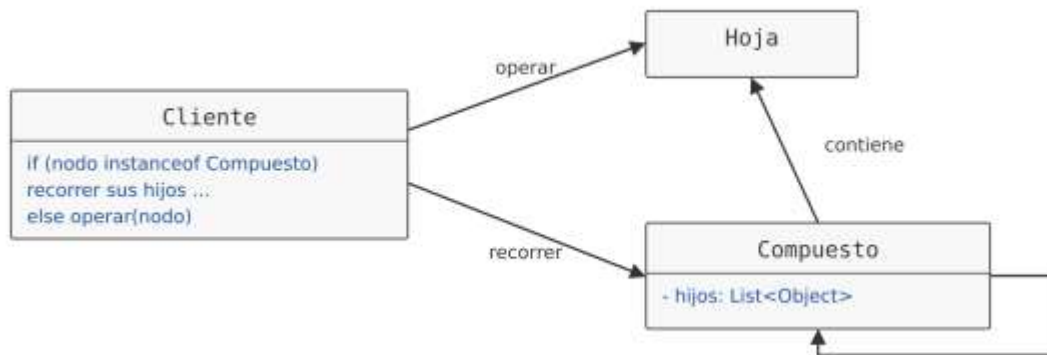
Los estructurales combinan objetos por composición, de modo que varios objetos se usen como uno solo.

Composite

El problema

Hay estructuras que son árboles de partes. Un dibujo tiene figuras y grupos, y un grupo tiene figuras y otros grupos. Un pedido tiene productos y combos, y un combo tiene productos y otros combos. Una carpeta tiene archivos y carpetas. En todos los casos hay dos tipos de nodo, el simple y el que contiene a otros, y el cliente quiere hacer lo mismo con los dos, dibujar, calcular el precio, calcular el tamaño.

Si el simple y el compuesto son clases sin nada en común, el cliente tiene que preguntar de qué tipo es cada nodo antes de operar. Aparece el *instanceof*, el *cast* y una rama por tipo, y esa cadena se repite en cada operación. Además el cliente tiene que recorrer el árbol por su cuenta, con una recursión propia por cada operación, porque el compuesto no sabe hacer nada con sus hijos.

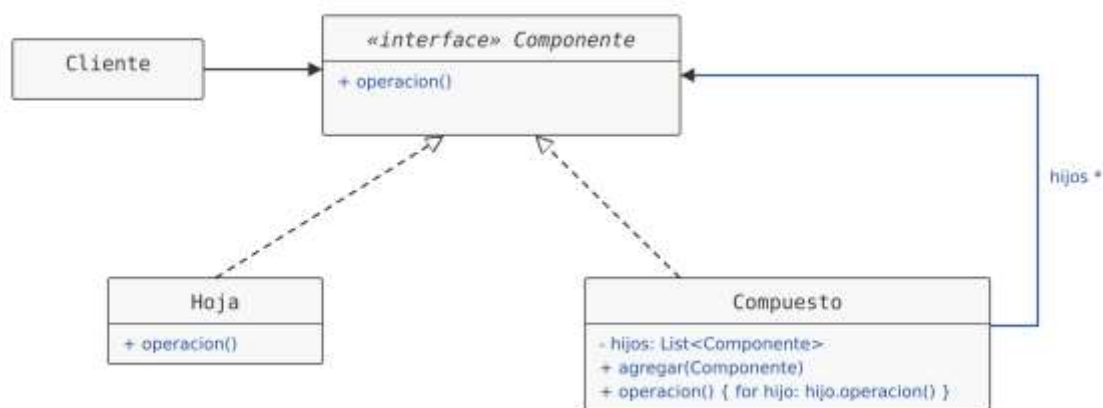


Sin el patrón. El cliente distingue el nodo simple del compuesto en cada operación, y recorre el árbol por su cuenta.

La solución

Se define una interfaz común, el Componente, con las operaciones que el cliente necesita. La Hoja la implementa haciendo el trabajo. El Compuesto también la implementa, pero guarda una lista de Componentes y resuelve cada operación recorriendo la lista y llamando a la misma operación en cada hijo. Como los hijos son Componentes, pueden ser hojas u otros compuestos, y la recursión sale sola.

Los participantes son tres, más el cliente. El Componente es la interfaz. La Hoja es el nodo sin hijos. El Compuesto es el nodo con hijos, que tiene además los métodos para agregarlos y quitarlos. El Cliente trabaja siempre con Componente y nunca pregunta cuál de los dos tiene adelante.



Estructura de Composite. Hoja y Compuesto implementan la misma interfaz, y el Compuesto delega cada operación a sus hijos, que son Componentes.

Hay una decisión de diseño que el libro discute. Los métodos para agregar y quitar hijos pueden ir en la interfaz Componente, y entonces el cliente puede llamarlos sobre cualquier nodo aunque en una hoja no tengan sentido, o pueden ir solo en el Compuesto, y entonces el cliente tiene que saber que tiene un compuesto para poder agregarle algo. La primera opción



es más uniforme y la segunda es más segura. En esta pre clase se usa la segunda, porque una hoja que lanza una excepción al recibir *agregar()* es una clase que promete lo que no cumple.

Ejemplo

Un editor de dibujo tiene círculos y rectángulos, y permite agrupar figuras para moverlas juntas. Un grupo puede contener otros grupos. La interfaz *Figura* declara lo que el editor hace con cualquier cosa que esté en el lienzo:



```
public interface Figura {
    void dibujar(Lienzo lienzo);
    void mover(int dx, int dy);
}

public class Circulo implements Figura {

    private int x;
    private int y;
    private final int radio;

    public Circulo(int x, int y, int radio) {
        if (radio <= 0) {
            throw new IllegalArgumentException("El radio debe ser mayor a cero");
        }
        this.x = x;
        this.y = y;
        this.radio = radio;
    }

    @Override
    public void dibujar(Lienzo lienzo) {
        lienzo.circulo(this.x, this.y, this.radio);
    }

    @Override
    public void mover(int dx, int dy) {
        this.x += dx;
        this.y += dy;
    }
}

public class Grupo implements Figura {

    private final List<Figura> hijas = new ArrayList<>();

    public boolean agregar(Figura figura) {
        if (figura == null || figura == this) {
            return false;
        }
        this.hijas.add(figura);
        return true;
    }

    @Override
    public void dibujar(Lienzo lienzo) {
        for (Figura hija : this.hijas) {
            hija.dibujar(lienzo);
        }
    }

    @Override
    public void mover(int dx, int dy) {
        for (Figura hija : this.hijas) {
            hija.mover(dx, dy);
        }
    }
}
```



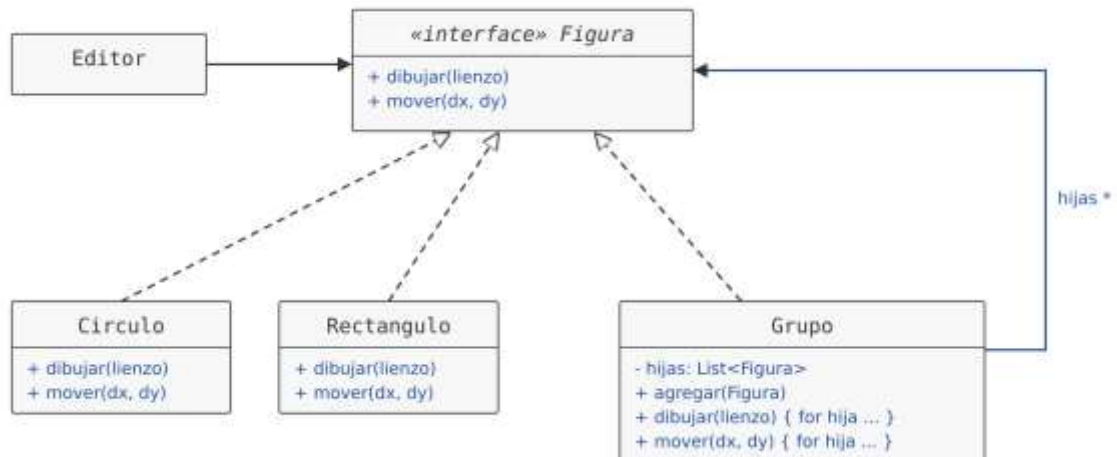
El editor arma un logo con dos círculos y un rectángulo, lo mete en un grupo, y ese grupo dentro del dibujo completo. Mover el dibujo es una sola llamada:

```
Grupo logo = new Grupo();
logo.agregar(new Circulo(10, 10, 5));
logo.agregar(new Circulo(30, 10, 5));
logo.agregar(new Rectangulo(10, 20, 25, 8));

Grupo dibujo = new Grupo();
dibujo.agregar(logo);
dibujo.agregar(new Circulo(80, 80, 20));

dibujo.mover(15, 0);    // mueve el logo entero y el circulo suelto
dibujo.dibujar(lienzo);
```

El método *mover()* de *Grupo* no sabe si cada hija es un círculo o un grupo con veinte figuras adentro. Llama a *mover()* y cada una resuelve. El editor tampoco lo sabe, y no necesita saberlo.



El ejemplo con la estructura de Composite. *Circulo* y *Rectangulo* son hojas, *Grupo* es el compuesto, y el editor trabaja con *Figura*.

Ventajas y desventajas

- El cliente trata igual a un objeto y a un árbol de objetos. No hay *instanceof* ni recorridos escritos a mano.
- Agregar un tipo de hoja o de compuesto es agregar una clase que implementa Componente. El cliente y el Compuesto no cambian.
- Es fácil que la interfaz Componente quede demasiado general, con operaciones que en alguna hoja no significan nada. Cuanto más distintos son los nodos, más forzada queda la interfaz.
- Un compuesto puede contener lo que sea, incluso a sí mismo por error. Las restricciones sobre qué puede ir adentro de qué hay que escribirlas a mano, como el chequeo de *agregar()*.



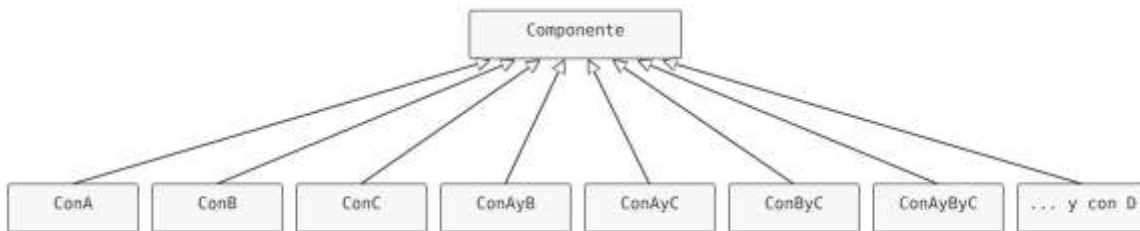
Composite. La hoja y el compuesto implementan la misma interfaz, y el compuesto delega en sus hijos.

Decorator

El problema

Un objeto necesita responsabilidades adicionales, y cuáles tiene cambia en tiempo de ejecución. Un personaje puede llevar espada, escudo y armadura, en cualquier combinación, y ganarlos o perderlos durante la partida. Un flujo de datos puede comprimirse, cifrarse y registrarse. Un café puede llevar leche, crema y chocolate.

La primera solución es la herencia, una subclase por variante. Con tres extras opcionales son siete subclases, con cuatro son quince, y cada combinación nueva es una clase más. Además la variante se elige al escribir el *new* y no se puede cambiar después, así que un personaje que pierde el escudo tiene que convertirse en un objeto de otra clase. La segunda solución es meter todos los extras como atributos booleanos en la clase base, que entonces conoce todos los extras posibles y cambia con cada uno nuevo.

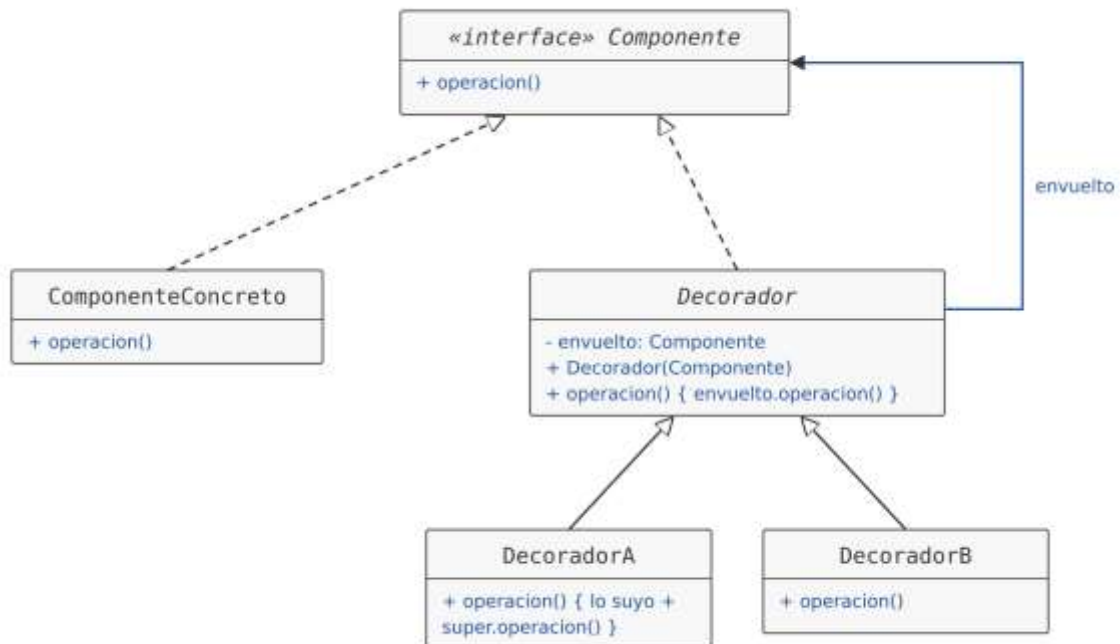


Sin el patrón. Una subclase por combinación de extras, elegida al crear el objeto y fija desde entonces.

La solución

Cada responsabilidad adicional se escribe en una clase propia, el Decorador, que implementa la misma interfaz que el objeto al que envuelve y guarda una referencia a él. Por defecto el Decorador delega cada operación al objeto envuelto. Un decorador concreto sobrescribe las operaciones que le interesan, hace su parte y llama a la del objeto envuelto. Como el decorador y el objeto tienen la misma interfaz, un decorador puede envolver a otro decorador, y los extras se apilan.

Los participantes son cuatro. El Componente es la interfaz común. El Componente Concreto es el objeto base, el que hace el trabajo de fondo. El Decorador es la clase abstracta que implementa Componente y guarda el Componente envuelto. Los decoradores concretos son sus subclases, una por responsabilidad.



Estructura de Decorator. El Decorador implementa la misma interfaz que envuelve, así que se puede apilar sobre el objeto base o sobre otro decorador.

Ejemplo

En un juego, un *Personaje* tiene ataque y defensa. El guerrero tiene valores base, y cada pieza de equipamiento suma a uno de los dos. Un personaje puede tener varias piezas, y puede perderlas:



```
public interface Personaje {
    int ataque();
    int defensa();
}

public class Guerrero implements Personaje {

    @Override
    public int ataque() {
        return 10;
    }

    @Override
    public int defensa() {
        return 5;
    }
}

public abstract class Equipamiento implements Personaje {

    private final Personaje base;

    public Equipamiento(Personaje base) {
        if (base == null) {
            throw new IllegalArgumentException("El equipamiento necesita un personaje");
        }
        this.base = base;
    }

    @Override
    public int ataque() {
        return this.base.ataque();    // por defecto, delega
    }

    @Override
    public int defensa() {
        return this.base.defensa();
    }
}

public class Espada extends Equipamiento {

    public Espada(Personaje base) {
        super(base);
    }

    @Override
    public int ataque() {
        return super.ataque() + 8;    // Lo suyo, mas lo del envuelto
    }
}

public class Escudo extends Equipamiento {

    public Escudo(Personaje base) {
        super(base);
    }

    @Override
    public int defensa() {
```

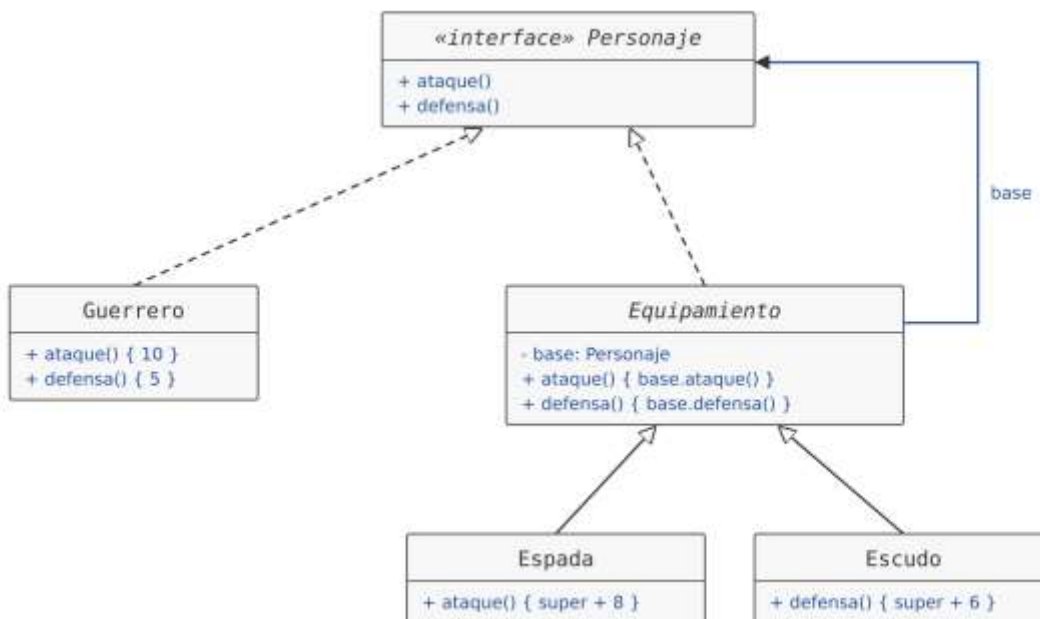


```
    return super.defensa() + 6;  
  }  
}
```

Equipar es envolver. Un guerrero con espada y escudo es un *Escudo* que envuelve a una *Espada* que envuelve a un *Guerrero*, y el resto del juego lo trata como un *Personaje* más:

```
Personaje heroe = new Guerrero();  
heroe = new Espada(heroe);  
heroe = new Escudo(heroe);  
  
heroe.ataque();    // 10 + 8 = 18  
heroe.defensa();   // 5 + 6 = 11
```

Perder el escudo es volver a la referencia anterior, la *Espada*. Ninguna clase se llama *GuerreroConEspadaYEscudo*, y una armadura nueva es una clase con un solo método.



El ejemplo con la estructura de Decorator. Espada y Escudo envuelven a un Personaje, que puede ser el Guerrero o a su vez otro equipamiento.

Ventajas y desventajas

- Las responsabilidades se agregan y se quitan en tiempo de ejecución, en cualquier combinación, sin una clase por combinación.
- Cada extra vive en su propia clase chica, con una sola razón para cambiar.
- Un objeto muy decorado es una cadena de objetos, y al depurar hay que atravesarla. Y como todos son del mismo tipo, no se puede preguntar “¿tiene escudo?” sin recorrer la cadena.



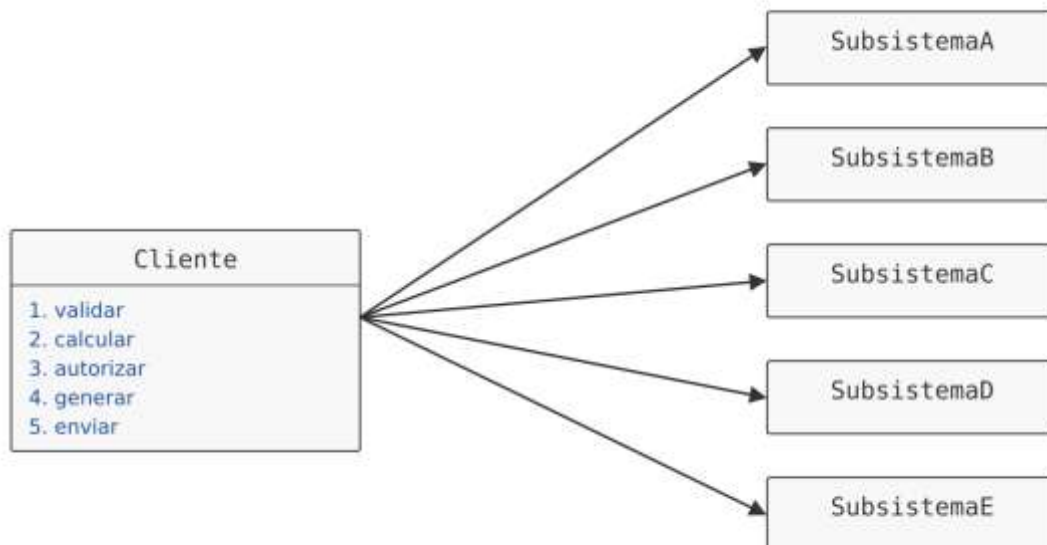
- Los decoradores no son el objeto base. Si el resto del programa guardó una referencia al *Guerrero* antes de equiparlo, esa referencia no ve la espada.

Decorator. Un objeto que implementa la misma interfaz que envuelve, hace su parte y delega el resto.

Facade

El problema

Una operación del negocio necesita coordinar varias clases de un subsistema, en un orden determinado y pasando datos de una a otra. Emitir una factura es validar el cliente, calcular impuestos, pedir autorización, generar el archivo y enviarlo. Si esa coordinación la escribe el cliente, el cliente conoce las cinco clases, sabe el orden, y repite la secuencia en cada lugar donde se emite una factura. Cualquier cambio en el subsistema, una clase que se divide en dos, un paso nuevo, obliga a abrir todos esos lugares.

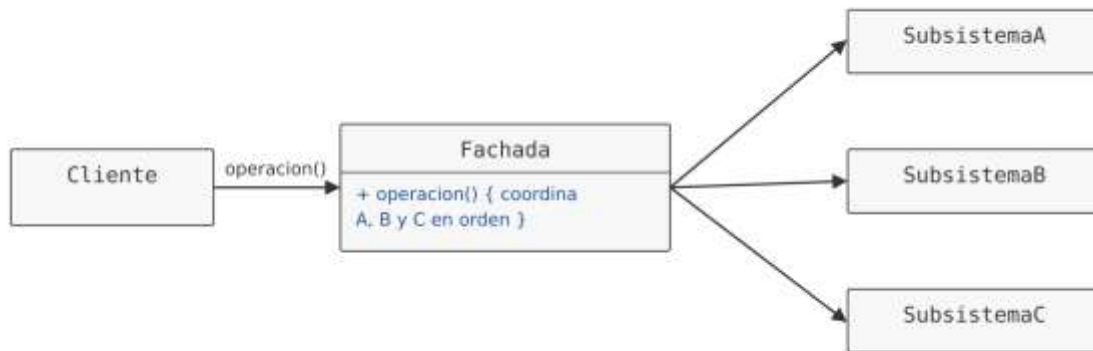


Sin el patrón. El cliente conoce las cinco clases del subsistema, el orden de las llamadas y qué dato pasa de una a otra.

La solución

Se escribe una clase, la Fachada, con un método por operación del negocio. La Fachada conoce las clases del subsistema y hace la coordinación adentro. El cliente conoce solo a la Fachada y le pide la operación completa con una llamada. El subsistema no conoce a la Fachada ni cambia por ella.

Los participantes son tres. La Fachada, con los métodos de alto nivel. Las clases del subsistema, que hacen el trabajo y siguen pudiendo usarse directamente si alguien necesita un control más fino. Y el Cliente, que habla con la Fachada.



Estructura de Facade. La Fachada conoce el subsistema y ofrece una operación por llamada. El cliente no conoce el subsistema.

Es el patrón más simple del catálogo, y por eso mismo el más usado sin nombrarlo. En una aplicación con modelo, vista y controlador, el modelo suele exponer una clase de este tipo, la partida, la caja, la cuenta, y el controlador le pide operaciones enteras en vez de coordinar clases del modelo desde afuera.

Ejemplo

Emitir una factura electrónica requiere validar el CUIT del cliente, calcular el total con impuestos, pedir el código de autorización al servicio de la agencia de recaudación, generar el PDF y enviarlo por correo. Cada paso es una clase que ya existe. La fachada los coordina:



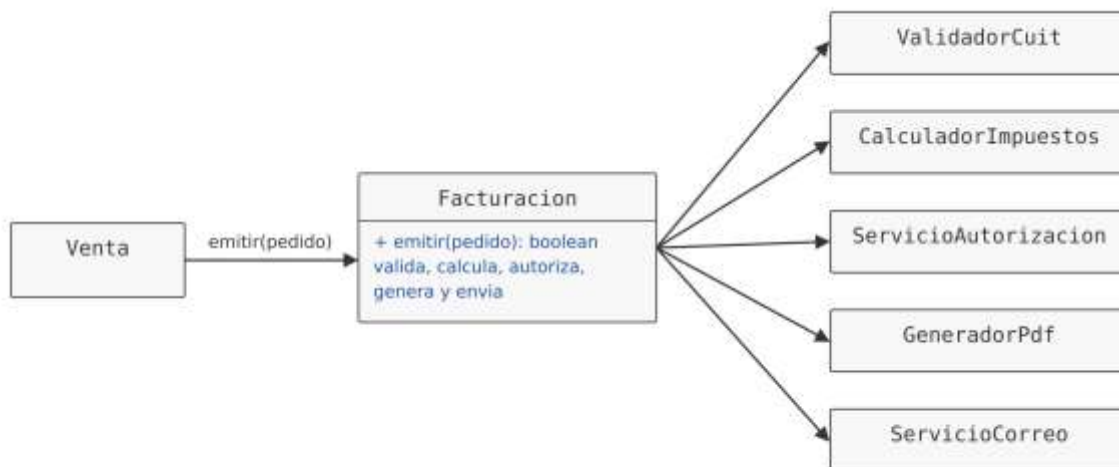
```
public class Facturacion {

    private final ValidadorCuit validador;
    private final CalculadorImpuestos impuestos;
    private final ServicioAutorizacion autorizacion;
    private final GeneradorPdf pdf;
    private final ServicioCorreo correo;

    public Facturacion(ValidadorCuit validador, CalculadorImpuestos impuestos,
        ServicioAutorizacion autorizacion, GeneradorPdf pdf, ServicioCorreo
correo) {
        this.validador = validador;
        this.impuestos = impuestos;
        this.autorizacion = autorizacion;
        this.pdf = pdf;
        this.correo = correo;
    }

    public boolean emitir(Pedido pedido) {
        if (!this.validador.esValido(pedido.getCuit())) {
            return false;
        }
        double total = this.impuestos.totalConImpuestos(pedido);
        String codigo = this.autorizacion.solicitar(pedido.getCuit(), total);
        if (codigo == null) {
            return false;
        }
        byte[] archivo = this.pdf.generar(pedido, total, codigo);
        return this.correo.enviar(pedido.getCorreoCliente(), archivo);
    }
}
```

Quien vende escribe una línea, *facturacion.emitir(pedido)*, y recibe si salió o no. No importa ninguna de las cinco clases, no conoce el orden y no sabe que hay un código de autorización. Si mañana la autorización se pide antes de calcular el PDF con otro formato, cambia *emitir()* y nada más. Las cinco clases entran por constructor, como cualquier dependencia, así que una prueba puede pasarle un servicio de autorización falso y verificar *emitir()* sin conectarse a nada.



El ejemplo con la estructura de Facade. Facturacion coordina las cinco clases, y quien vende solo llama a emitir().

Ventajas y desventajas

- El cliente depende de una clase en vez de cinco, y la secuencia de la operación está escrita una vez.
- El subsistema puede cambiar por dentro sin que el cliente se entere, siempre que la Fachada conserve su método.
- Una fachada que crece sin criterio se convierte en una clase que sabe de todo, con decenas de métodos que no tienen relación entre sí.
- Esconde el subsistema pero no lo simplifica. Si la coordinación es intrincada, sigue siéndolo adentro de la Fachada, solo que en un único lugar.

Facade. Una clase con una operación por llamada, que coordina el subsistema para que el cliente no tenga que hacerlo.

En la biblioteca

Los tres están en Swing y en Java desde siempre. Un *JPanel* dentro de un *JPanel* dentro de una ventana es Composite. *Container* es el compuesto, cada componente es la hoja, y cuando la ventana se dibuja llama a `paint()` en cada hijo, que si es un contenedor lo llama en los suyos. Un *JScrollPane* alrededor de una *JTable* es Decorator, un componente que envuelve a otro y le agrega las barras de desplazamiento. En *java.io* es la forma de trabajo, `new BufferedReader(new FileReader(archivo))` es un lector que envuelve a otro y le agrega el buffer. Y `Files.readString(ruta)` es Facade, una llamada que por dentro abre el archivo, crea el lector, lee, decodifica y cierra.

Un panel con paneles es Composite, un scroll alrededor de una tabla es Decorator, y leer un archivo en una línea es Facade.



Para el proyecto

Las tres preguntas de esta pre clase son si hay algo que se agrupa y el grupo se trata como una unidad, una escuadra de unidades, un inventario con bolsas adentro; si hay atributos que un objeto gana y pierde durante la partida, equipamiento, efectos, mejoras; y si el controlador llama a más de dos o tres clases del modelo para completar una acción. La primera lleva a Composite, la segunda a Decorator y la tercera a Facade, y la respuesta a la tercera suele ser que la partida es la fachada del modelo.

Bibliografía sugerida para profundizar

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

- Capítulo 4, “Structural Patterns”, con “Composite”, “Decorator” y “Facade”. El ejemplo de Composite es un editor gráfico, y el de Decorator, un componente de interfaz con borde y scroll.

Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2.^a ed.). O'Reilly.

- Capítulo 3, “The Decorator Pattern”, con la cafetería y después los flujos de *java.io*; capítulo 7, “The Adapter and Facade Patterns”, con el cine en casa; y capítulo 9, “The Iterator and Composite Patterns”, con el menú de un restaurante.

Shvets, A. (2020). *Dive Into Design Patterns*. Refactoring.Guru. Versión web gratuita en <https://refactoring.guru/es/design-patterns>

- La sección “Patrones estructurales”, con el problema, la estructura, los diagramas y el código en Java de cada uno de los siete.

Oracle. *The Java Tutorials*, “How to Use Scroll Panes” y “Buffered Streams”.

- Los dos decoradores de la biblioteca que se usan sin nombrarlos, el panel con scroll y los flujos con buffer.